

UIOX ISA Reference

Readable kernel-focused architecture document for ARM64, ARM32, RISC-V 64, and x86-64

Overview

This document is a readable ISA reference for the UIOX project across its four target architectures: ARM64, ARM32, RISC-V 64, and x86-64. It is not a replacement for the vendor architecture manuals. Instead, it defines the kernel-facing architectural contract that UIOX needs: boot entry conventions, privilege transitions, syscall ABI, trap handling, context-save expectations, MMU control, TLB maintenance, barriers, atomics, idle instructions, and compiler constraints.

The goal is practical portability. A UIOX kernel engineer should be able to use this document to understand what low-level assumptions generic kernel code may rely on, what must remain architecture-specific, and how the four ports line up with one another.

Architecture Set

UIOX Name	ISA Family	Kernel Mode	Typical Toolchain	Key Traits
arm64	ARMv8-A AArch64	EL1	aarch64-elf-gcc	TTBR0/TTBR1, VBAR_EL1, SVC, ERET
arm32	ARMv7-A AArch32	SVC mode	arm-none-eabi-gcc	TTBR0/TTBR1, VBAR, SVC, exception return
riscv64	RV64IMAFDC + Zicsr + Zifencei	S-mode	riscv64-elf-gcc	satp, stvec, ecall, sret, sfence.vma
x8664	AMD64 long mode	Ring 0	x8664-elf-gcc	CR3, IDT, SYSCALL, SYSRETQ, IRETQ

Cross-Architecture UIOX Contract

- Every architecture port must provide boot handoff into common kernel code, including stack initialization and transfer into `uioxkernelmain` or the architecture-specific early equivalent.
- Every port must support user-to-kernel syscall entry, interrupt and exception entry, and a correct return path back to the previous privilege level.
- Every port must provide address-space installation, page-table switching, and TLB invalidation through architecture hooks rather than open-coded generic logic.
- Every port must define a context-switch save set sufficient for preemptive scheduling, interrupt recovery, and resume into user or kernel execution.
- Every port must expose barriers, atomics, and a CPU idle primitive through common abstractions so that higher-level kernel code stays portable.

ARM64

Execution Model

ARM64 in UIOX targets AArch64 under ARMv8-A. User code runs at EL0 and the kernel runs at EL1. Exceptions from user or kernel space land through the EL1 vector table, and architectural return from an exception uses ERET.

Boot Entry Convention

Register	Meaning at Kernel Entry
x0	device-tree physical address (dtbpa)
x1	boot argument block physical address (argspa)
x2	reserved / zero
x3	reserved / zero

General Register Roles

Registers	Role
x0-x7	arguments and return values
x8	syscall number convention in UIOX
x9-x15	caller-saved temporaries

Registers	Role
x16-x17	intra-procedure scratch
x19-x28	callee-saved general registers
x29	frame pointer
x30	link register
sp	stack pointer

System Registers Relevant to UIOX

Register	Purpose
VBAR_EL1	exception vector base
SCTLR_EL1	MMU, cache, and control bits
TTBR0_EL1	user translation base
TTBR1_EL1	kernel translation base
TCR_EL1	translation configuration
MAIR_EL1	memory attribute encodings
ESR_EL1	exception syndrome
FAR_EL1	fault address
ELR_EL1	saved return PC
SPSR_EL1	saved processor state
DAIF	interrupt mask bits
CNTFRQ_EL0 and CNTP_*	architected timer interface

Key Instruction Families

- Syscall entry uses SVC, and exception return uses ERET.
- Address-space and translation management use TTBR0_EL1, TTBR1_EL1, TLBI operations, DSB, and ISB.
- Memory ordering uses DMB, DSB, and ISB depending on whether ordering, completion, or context synchronization is required.
- Idle and event instructions are WFI, WFE, and SEV.
- Atomic operations are provided through load-exclusive and store-exclusive pairs such as LDXR/STXR, and optionally newer architectural atomics when enabled.

Syscall ABI

UIOX on ARM64 uses x8 for the syscall number and x0 through x5 for arguments. User code traps with SVC #0. Kernel entry code saves a trap frame, dispatches by the x8 syscall number, returns the result in x0, and resumes with ERET.

Context Switch and MMU Notes

A practical ARM64 context switch saves the callee-saved register set, stack pointer, saved exception state when returning to user space, and the active user translation base. UIOX should treat TTBR0_EL1 as part of the current process address-space identity and keep kernel mappings stable through TTBR1_EL1.

ARM32

Execution Model

ARM32 in UIOX targets ARMv7-A in AArch32 state. User code runs in User mode, while the kernel typically runs in SVC or System mode. Interrupt and fault handling may use banked state depending on exception type. Kernel code in this project assumes ARM mode rather than Thumb-only operation.

Boot Entry Convention

Register	Meaning at Kernel Entry
r0	reserved / zero
r1	reserved / zero
r2	device-tree physical address (dtbpa)

Register	Meaning at Kernel Entry
r3	boot argument block physical address (argspa)

General Register Roles

Registers	Role
r0-r3	arguments, return values, scratch
r4-r11	callee-saved general registers
r12	intra-procedure scratch
r13	stack pointer
r14	link register
r15	program counter
r7	syscall number convention in UIOX

System Registers Relevant to UIOX

Register	Purpose
VBAR	exception vector base
SCTLR	MMU, cache, and control bits
TTBR0	user translation base
TTBR1	kernel translation base
DACR	domain access control
DFSR / IFSR	fault status
DFAR / IFAR	fault address
CPSR / SPSR	processor state
TPIDRURW	thread pointer / TLS hook

Key Instruction Families

- Syscall entry uses SVC #0 and returns through the architecture-specific exception return path after restoring state.
- MMU and TLB control use CP15 register access, TTBR writes, and invalidate operations.
- Memory ordering and synchronization use DMB, DSB, and ISB.
- Idle and event operations use WFI, WFE, and SEV.
- Atomic primitives rely on LDREX/STREX style exclusive accesses.

Portability Note for UIOX

ARM32 is especially important to generic-code portability because pointers are 32-bit. Code that casts between pointers and integer storage carelessly may compile on 64-bit targets but fail under strict ARM32 warnings. The UIOX project has already encountered this pattern and should treat `uintptr_t`-based portability as a documented rule.

RISC-V 64

Execution Model

UIOX on RISC-V targets RV64 with user mode in U-mode and the kernel in S-mode. Machine mode is assumed to belong to firmware or a supervisor runtime such as OpenSBI. Traps into the kernel land through STVEC, and architectural return from supervisor trap handling uses SRET.

Required ISA Profile

The practical baseline for the project is RV64IMAFDC with Zicsr and Zifencei. In the build history this appears as `rv64imafdczicsr_zifencei`. For kernel engineering, the most important pieces are integer execution, atomics, CSR access, and instruction or translation fencing.

Boot Entry Convention

Register	Meaning at Kernel Entry
a0	device-tree physical address (dtbpa)
a1	boot argument block physical address (argspa)

Integer Register Roles

Registers	Role
a0-a7	arguments and return values, with a7 used for syscall number
s0-s11	saved registers across calls
t0-t6	temporaries
sp	stack pointer
ra	return address
tp	thread pointer

Supervisor CSRs Relevant to UIOX

CSR	Purpose
stvec	trap vector base
sstatus	supervisor status
sepc	saved return PC
scause	trap cause
stval	faulting address or associated value
satp	page-table root and address-space identity
sscratch	trap scratch state
sie / sip	interrupt enable and pending state

Key Instruction Families

- Syscall entry uses ECALL and trap return uses SRET.
- Address-space switching uses SATP and TLB invalidation uses SFENCE.VMA.
- Memory ordering uses FENCE and instruction synchronization uses FENCE.I.
- Idle uses WFI.
- Atomics use LR/SC pairs and AMO instructions.

Syscall ABI

UIOX on RISC-V uses a7 for the syscall number and a0 through a5 for arguments. User code enters the kernel with ECALL. The supervisor trap path distinguishes user ECALL from other traps via SCAUSE, advances SEPC appropriately, returns the result in a0, and resumes with SRET.

x86-64

Execution Model

UIOX on x86-64 targets AMD64 long mode, with user execution in ring 3 and kernel execution in ring 0. Fast syscall entry uses SYSCALL and fast return uses SYSRETQ. Interrupts and faults return with IRETQ.

Boot Entry Convention

Register	Meaning at Kernel Entry
rdi	boot argument block physical address (argspa)
rsi	device-tree or platform-description physical address (dtbpa)

General Register Roles

Registers	Role
rax	return value and syscall number
rdi, rsi, rdx, rcx, r8, r9	standard SysV argument registers
rdi, rsi, rdx, r10, r8, r9	syscall argument registers
rbx, rbp, r12-r15	callee-saved registers

Registers	Role
rsp	stack pointer
rip	instruction pointer

Control State Relevant to UIOX

Register / MSR	Purpose
CR0, CR4	global CPU control state
CR2	page-fault address
CR3	page-table root
EFER	long-mode and syscall enable bits
STAR / LSTAR / SFMASK	syscall entry configuration
IDTR / GDTR	descriptor table bases
FS.base / GS.base	TLS or per-CPU state

Key Instruction Families

- Fast syscall entry uses SYSCALL and fast return uses SYSRETQ.
- Interrupt and fault return use IRETQ.
- Address-space switching uses CR3, and single-page invalidation uses INVLPG.
- Descriptor-table and task-state setup use LGDT, LIDT, and LTR.
- Idle uses HLT, while PAUSE is useful in spin loops.
- Atomic synchronization uses LOCK-prefixed operations such as CMPXCHG and XADD.

Compiler Constraints in UIOX

The project build flags for x86-64 disable the red-zone and disable MMX, SSE, and SSE2. This matters architecturally. Generic kernel code must not rely on floating-point or SIMD state, and interrupt-safe code must assume that memory below RSP is not preserved. The project has already seen one consequence of this rule when floating-point code had to be rewritten into fixed-point form.

Cross-Architecture Comparisons

Syscall ABI Summary

Architecture	Enter	Return	Syscall Number	Arguments
ARM64	SVC #0	ERET	x8	x0-x5
ARM32	SVC #0	exception return	r7	r0-r5
RISC-V 64	ECALL	SRET	a7	a0-a5
x86-64	SYSCALL	SYSRETQ	rax	rdi, rsi, rdx, r10, r8, r9

Address-Space Control Summary

Architecture	Primary Address-Space Register(s)	TLB Maintenance
ARM64	TTBR0_EL1 and TTBR1_EL1	TLBI operations followed by DSB and ISB
ARM32	TTBR0 and TTBR1	CP15 invalidate operations with barrier sequencing
RISC-V 64	SATP	SFENCE.VMA
x86-64	CR3	INVLPG or CR3 reload

Idle and Barrier Summary

Architecture	Idle	Ordering / Synchronization Notes
ARM64	WFI	DMB, DSB, ISB
ARM32	WFI	DMB, DSB, ISB
RISC-V 64	WFI	FENCE and FENCE.I
x86-64	HLT	stronger default ordering, plus MFENCE/LFENCE/SFENCE when needed

Project Rules for Portable Kernel Code

- Do not assume a pointer width wider than `uintptr_t`. Integer and pointer conversions must be written so they remain valid under both 32-bit and 64-bit targets.
- Do not require floating-point in generic kernel paths. If an algorithm can be expressed in fixed-point, that is the preferred portable form for UIOX kernel code.
- Do not open-code architecture-specific TLB, barrier, or trap-return logic inside generic subsystems. Those operations belong behind architecture hooks or portability macros.
- Keep syscall numbering, trap-frame semantics, and process-address-space semantics aligned across ports even when the raw register conventions differ.
- Treat this document as the UIOX kernel contract, while using vendor manuals for full instruction encodings and detailed architectural corner cases.

External Reference Anchors

Architecture	Reference
ARM64	https://developer.arm.com/documentation/ddi0487/gb
ARM32	https://www.cl.cam.ac.uk/research/srg/han/ACS-P35/zynq/ARMv7-A-R-manual.pdf
RISC-V 64	https://docs.riscv.org/reference/isa/v20260120/priv/priv-index.html
x86-64	Use the AMD64 Architecture Programmer's Manual volumes from AMD as the canonical ISA reference.

Recommended next expansion

- The next strong version of this document should add exact trap-frame layouts, exact context-switch save sets, MMU hook APIs, and one common syscall-number policy table for all ports.